CS2201 Computer Organization & Architecture

L25: Advanced Cache Optimization Techniques-2



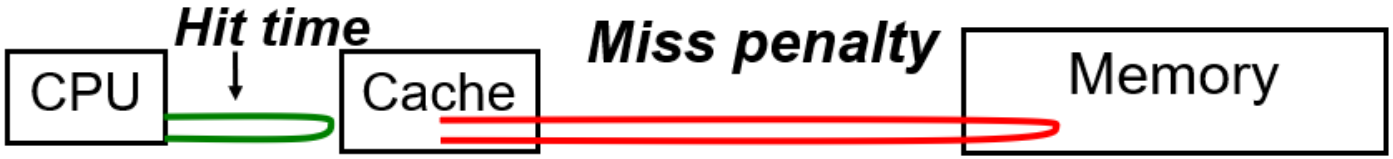
John Jose

Associate Professor

Department of Computer Science & Engineering

Indian Institute of Technology Guwahati

How to optimize cache ?

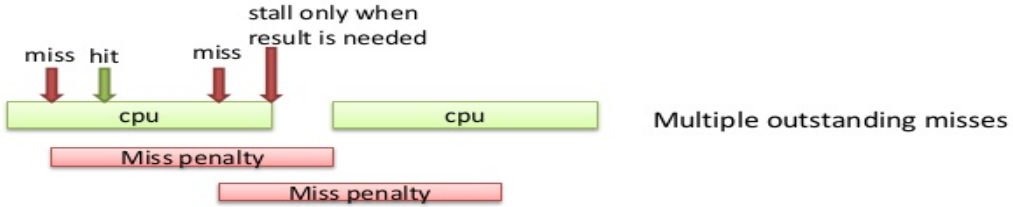


Average memory access time = Hit time + (Miss rate×Miss penalty)

- Motives
  - Reducing the miss rate
  - Reducing the miss penalty
  - Reducing the hit time

Non-blocking Caches

- **Non blocking caches to increase cache bandwidth**
- Caches can serve hits under multiple cache miss in progress
  - *(a) Hit under miss (b) Hit under multiple miss*



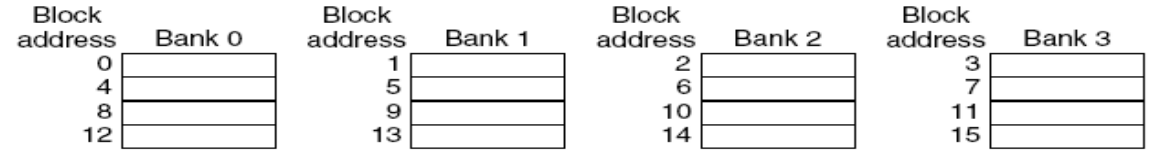
- Must needed for OOO superscalar processor for IPC increase
- L2 must support it with L1-MSHR (Miss Status Holding Reg.)
- On an L1 miss allocate MSHR entry, clear upon L2 respond with cache block reply.

Non-blocking Caches

- **Non blocking caches to increase cache bandwidth**
- Processors can hide L1 miss penalty but not L2 miss penalty
- Reduces the effective miss penalty by overlapping miss latencies
- Significantly increases the complexity of the cache controller as there can be multiple outstanding memory accesses
- Requires pipelined or banked memory system

Multi-banked Caches

- **Multi-banked caches to increase cache bandwidth**
- Rather having a single monolithic block, divide cache into independent banks that can support simultaneous accesses.
  - ARM Cortex-A8 supports 1-4 banks for L2
  - Intel i7 supports 4 banks for L1 and 8 banks for L2
- Sequential Interleaving



Four-way interleaved cache banks using block addressing. Assuming 64 bytes per blocks, each of these addresses would be multiplied by 64 to get byte addressing.

Early Restart

- **Early restart to reduce miss penalty**
- Do not wait for entire block to be loaded for restarting CPU
- **Early restart**
  - Request words in normal order
  - Send missed work to the processor as soon as it arrives
  - L2 controller is not involved in this technique

- **Early restart**



Critical Word First

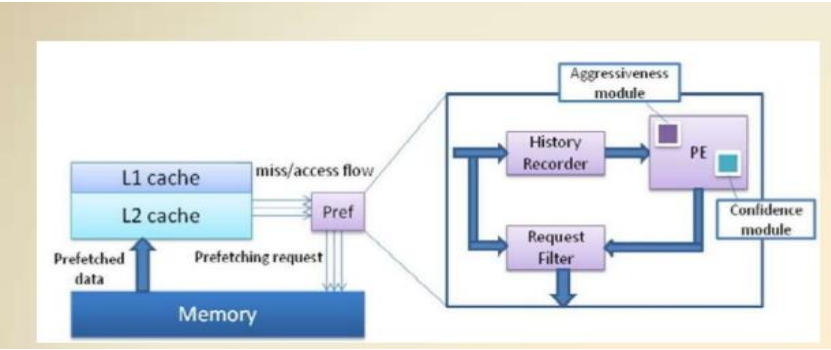
- **Critical word first to reduce miss penalty**
- **Critical word first**
  - Request missed word from memory first
  - Send it to the processor as soon as it arrives
  - Processor resume while rest of the block is filled in cache
  - L2 cache controller forward words of a block out of order.
  - L1 cache controller should re-arrange words in block
    - **Critical word first**

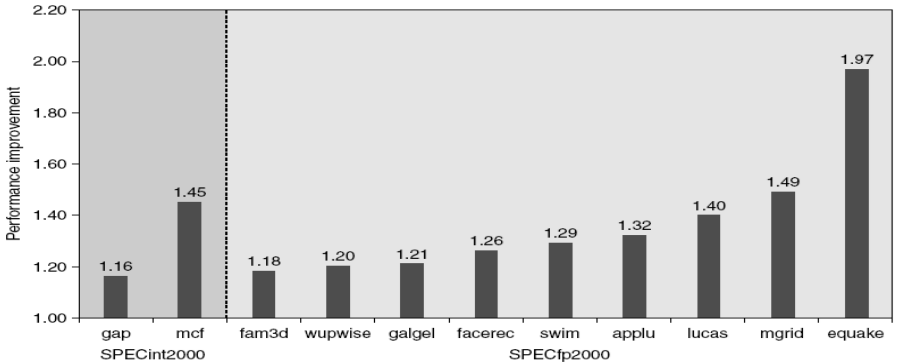


As soon as 3 is brought it is forwarded to CPU
In background 4, 1 and 2 is brought

Hardware Prefetching

- **Pre-fetching to reduce miss rate and miss penalty.**
- Pre-fetch items before processor request them.
- Fetch more blocks on miss -include next sequential block
- Requested block is kept in I-cache and next in stream buffer.
- If a missed block is in stream buffer, cache miss is cancelled





Match the Following

Match the following from Set X to unique element(s) in Set Y.

Set X	Set Y
(a) No-write allocate cache	1. exploits sequential interleaving
(b) Non-blocking cache	2. allows faster clock cycle time
(c) Pipelined cache	3. refers to dirty bit during eviction
(d) Multi-banked cache	4. use inclusive or exclusive hierarchy
(e) Hardware pre-fetching	5. reduces miss rate
(f) Victim cache	6. reduces miss penalty
(g) Way prediction	7. implementation needs MSHR
(h) Multi level cache	8. reduces hit time
(i) Write back cache	9. no block eviction on a write miss

a→9, b→7, c→2, d→1/8, e→5, f→6, g→8, h→4/6, i→3



johnjose@iitg.ac.in

http://www.iitg.ac.in/johnjose/

https://www.linkedin.com/in/john-jose-bb8431106/